

VesSkel

Vessel Skeletonization and Graph-Based Phenotype Analysis in Retinal Fundus Images

Simon Wittmann

Supervisor: Anna Möller

23. Juni 2026

Implementation Progress

1. preprocessing: binary closing & hole filling
2. new features & updated Feature Comparison Table
3. 3D thinning performance ($\rightarrow$ parallel scanning, epoch removal tracking)
4. benchmark results (2D + 3D vs skimage & VesselVio)
5. nicer CLI UX (shorter subcommands, worker feedback)
6. paper draft started
7. prediction notebook refactored

Preprocessing: Binary Closing & Hole Filling

Binary closing bridges small gaps in segmented vessels before thinning

Hole filling fills enclosed background regions inside vessels

Config Key	Default	Description
closing_iterations	0	binary closing iterations (0 = off)
fill_holes	false	fill enclosed background regions
max_hole_size	0	max filled hole area (0 = unlimited)
show_preprocessed	false	show preprocessed binary in napari

Supporting tests: in `test_preprocessing.py` covering closing, hole filling, size thresholding.

New Features

Key additions since last update:

1. Mean segment volume ($V = \pi \sum r_i^2$ for each segment)
2. Mean segment surface area ($A = 2\pi \sum r_i$)
3. Added to `per_segment_radii` output and summary features
4. Used in downstream classification

Feature Table Update:

1. Re-mapped which features are feasible (many marked “nope”)
2. only TODO left: per-node metrics

SHOW TABLE

3D Thinning Performance Improvements

Epoch-Based Removal Tracking

Problem: vanilla Lee94 re-checks simplicity (DFS on 26-neighbor graph) for **every** candidate before removal

Insight: `_mark_removable_candidates` already verified all candidates before any removals in this batch. If none of a candidate's 26 neighbors have been removed yet, the old verdict is still valid - skip the DFS.

Mechanism:

1. **epoch** increments per batch; stamped at each removed voxel
2. Scan 26 neighbors for `stamp == current epoch` (cheap, 26 int comparisons)
3. If no neighbor stamped → remove immediately
4. If neighbor stamped → re-run DFS to verify still simple
5. Never reset; monotonically increasing epoch makes old stamps invisible

Benchmark Results

2D (HRF dataset, 45 images)

	Mean (s)	vs vesskel
vesskel	0.084	-
skimage Zhang	0.157	1.87x slower
skimage Lee	0.438	5.21x slower
VesselVio Lee	0.796	9.46x slower

3D (vessap test volume)

	Mean (s)	vs vesskel
vesskel	0.730	-
skimage Lee	1.024	1.4x slower
VesselVio Lee	0.804	1.10x slower

Key Takeaways

1. 2D is highly optimized via LUT-based simplicity check
2. 3D competitive with skimage and VesselVio
3. Pure Python + Numba - no Cython, no compiled extensions

CLI Improvements

Shorter Subcommands

- `vesskel init config.json` instead of `vesskel config-init --out config.json`
- `vesskel validate config.json` instead of `vesskel validate-config --config config.json`

Instant Worker Feedback

- `vesskel run (-j N)` now prints: `Spawning N worker processes...`

Default Config Changed

- `branches`, `branch_text`, `summary` now default `false`

Paper Draft

First Table of Content draft started

also: Prediction Notebook Refactored

1. analysis/HRF_Prediction.ipynb restructured
2. Pipeline runs within the notebook
3. Cleaner separation between analysis and reporting
4. No notable Performance improvements, even though cleanup ran and more features available → more investigation needed

Summary & Next Steps

What I did:

1. Preprocessing (closing, hole filling)
2. Per-segment volume & surface area
3. 3D thinning speedups (parallel scanning, epoch tracking)
4. Benchmark scripts (2D + 3D)
5. CLI polish (shorter cmds, worker feedback)
6. Table of Content draft
7. Feature table finalized
8. Prediction notebook refactored

Next steps:

1. per_node features finalization
2. finalize Phenotype Prediction
3. nice documentation
4. Write the paper
5. (optional) more tests